

Canonical Rendering Sprint — the manual builder becomes a renderer over the scenario graph

Author: QA Architecture
Branch: feat/canonical-manual-renderer
Status: Ready for review — proven end-to-end on ONE capability (Sorting), zero regression elsewhere.
Revision: Incorporates the PR #321 architecture review — the canonical graph is kept **language- and presentation-neutral**; wording is owned by renderers; ordering is a first-class semantic assertion, not prose.

1. The problem this sprint fixes

The manual Test-Case-Lab output and the automation Script-Gen output are supposed to be two views of the same test. They were not. They were produced by **two independent generators** that could — and did — disagree.

The clearest evidence was the 20-case sorting export (test-cases-238.xlsx , REQ-005), which scored 2/10: every “Sort products” case rendered **checkout steps** (“Open checkout → fill First/Last/Zip → click Continue”) and a **CRUD expected result** (“The Sort products record is created successfully”). The requirement was about sorting; the output was about a form the app happened to expose.

Root cause (code-grounded, not guessed)

The scenario **graph** is already the single source of truth. It is consumed by **two** adapters:

Adapter	Consumer	What it read
toScriptGenSpecs()	Automation / Script Gen	node.actions + node.assertions (the canonical machine model)
toTestCaseLab()	Manual test cases	node.steps — legacy prose the old form-playbook builder guessed

So Script Gen rendered from the canonical actions, while the manual side copied whatever prose the legacy pickForm + CRUD-template path had produced. When a capability had no authored canonical actions, the legacy path fell back to “find any form on the app and fill it” — which is exactly how a sort scenario inherited checkout steps.

The architecture was ~80% built and stopped halfway. Only authentication had authored semantics / actionTemplate / assertionTemplate . Every other category (search, checkout, crud, payment, ...) had **zero** authored canonical content, so the manual side never had canonical actions to render and always fell back to the guessing path.

2. What this sprint does — complete the contract, don't add a layer

No new abstraction was introduced. We finished the design that already exists.

2.1 Phase 1 — the manual builder renders from canonical actions

`toTestCaseLab()` now renders each column from the canonical model **when it exists**, and falls back to the exact legacy output when it does not:

```
const manual = renderManualIfCanonical(n); // null when n.actions is empty → legacy
steps        : manual ? manual.steps      : n.steps.slice()
expectedResult : manual ? manual.expectedResult : n.expectedResult
testData      : manual ? manual.testData    : composeResolvedTestData(n)
```

The projection lives in a new **pure** module `src/graph/canonical-manual-renderer.ts` (`renderManualFromCanonical(actions, assertions, resolved?)`). It humanises canonical targets app-neutrally (`sort_dropdown` → “sort dropdown”), derives every sentence from the semantic fields, and fabricates no locators.

2.2 The canonical model stays presentation- and language-neutral

The graph owns **meaning**; renderers own **wording**. This is the architectural line the PR #321 review drew, and this revision holds it:

- The canonical model carries **zero prose**. There is no `ScenarioAction.description` and no `ScenarioAssertion.observable` — those fields mixed machine meaning with English presentation text, which would have leaked one renderer's wording (manual, English) into a model that also feeds BDD, Playwright, Java, Python, Coverage, RTM and Executive-Summary renderers.
- An **action** is purely `action / target / value`. Each renderer derives its own sentence from those semantics (the manual renderer says “In the sort dropdown, select the ‘Name (A to Z)’ option”; a Playwright renderer emits `selectOption`; a BDD renderer writes a `When` step).
- An **assertion** is a frozen `type + canonical target (+ optional expected)`. Where a check needs richer meaning (not richer wording), we add a **semantic primitive**, never a prose field — see ordering below.

2.3 New first-class assertion primitive — ordered

“The products are in alphabetical order” is genuine **business meaning**, not presentation. The frozen `AssertionType` vocabulary could not express it, so previously the intent could only have lived in a human `observable` string. The review's directive — “add one missing assertion type instead of storing human-readable ordering descriptions” — is implemented:

```
AssertionType += 'ordered' // 11 → 12 frozen types
ScenarioAssertion += { // populated only when type === 'ordered'
  collection?: string; // WHAT is ordered e.g. "products"
  direction?: 'ascending' | 'descending';
  orderBy?: string; // the dimension e.g. "name"
}
export type OrderDirection = 'ascending' | 'descending';
```

These are semantic fields — machine-checkable, language-neutral, renderer-agnostic. The manual renderer turns them into a sentence; Script Gen turns them into a real Playwright ordering check (§3.2). Both read the **same** three fields.

2.4 Phase 2 — author ONE capability: Sorting

`search-pos-sort` is now fully authored in the KB (`search catalog`), with **no prose in the KB**:

```
actionTemplate:                                (semantics only – wording derived per render-
er)
  navigate product_list
  select  sort_dropdown = "Name (A to Z)"
assertionTemplate (all afterAction = search-pos-sort.select.sort_dropdown):
  ordered product_list collection=products direction=ascending orderBy=name
  value  sort_dropdown = "Name (A to Z)"
  visible product_item (opt.)
  visible cart (opt.)
```

The ordering invariant now rides on a **checkable semantic assertion**, independent of how the requirement happens to be worded.

3. Proof — before vs after, from the REAL pipeline

3.1 Manual renderer — `scripts/canonical-render-proof.ts`

Runs the exact engine path (`planScenarios` → `buildDraftTestCases` → `buildDeterministicOutput` → `materialize templates` → `assembleScenarioGraph` → `toTestCaseLab`). The App Profile deliberately exposes **only a checkout form**, reproducing the original bug’s conditions.

BEFORE (legacy `node.steps` — the 2/10 artifact):

```
1. Open the page under test
2. Enter "Sophia" in the First Name field
3. Enter "Johnson" in the Last Name field
4. Enter "Sample Zip/Postal Code" in the Zip/Postal Code field
5. Click the Continue button
Expected: ✓ The Sort products record is created successfully. | ✓ A success
confirmation ... | ...
```

AFTER (canonical-rendered `toTestCaseLab` — every sentence derived from semantics):

```
1. Open the product list page
2. In the sort dropdown, select the "Name (A to Z)" option
Expected Result:
  ✓ The products are displayed in ascending order by name.
  ✓ The sort dropdown holds the value "Name (A to Z)".
  ✓ The product item is displayed.
  ✓ The cart is displayed.
Test Data: sort dropdown: Name (A to Z)
```

The ordering line is produced by the renderer from `collection=products / direction=ascending / orderBy=name` — the KB stores none of that English.

Zero-regression check (same run): a non-authored scenario (`search-pos-match`) is byte-identical between `node` and projection — `steps unchanged=true`, `expected unchanged=true` .

3.2 Script Gen honestly handles `ordered` — `scripts/canonical-ordered-scriptgen-proof.ts`

The second consumer of the same assertion. `emitGraphAssertionLines` previously continued on unknown types (a silent drop). It now emits a **real ordering check** from the same three semantic fields:

```
const _ordered0 = (await page.getByLabel('products')).allTextContents().map((t) =>
  t.trim());
expect(_ordered0).toEqual([..._ordered0].sort((x, y) => x.localeCompare(y)));
```

`direction=descending` flips the comparator to `(x, y) => y.localeCompare(x)` . The proof asserts the lines are present and fails loudly if the type is ever dropped again.

4. Validation

- `npx tsc --noEmit` — **clean** (exit 0).
- Two proof harnesses — manual before/after + zero-regression, and Script-Gen ordering emission.
- Graph / adapter / builder / planner / renderer suites — all green
(`scenario-graph` 22, `scenario-graph-adapters` 18, `scenario-builder` 28, `scenario-planner` 53, `scenario-correctness` 25, `scenario-integrity` 32, `scenario-specific-test-data` 16, `canonical-test-case` 35, `canonical-test-data` 33, `script-gen-scenario-fidelity` 29, `feature-grounding-engine` / `expected-result-validator` / `intent-step-generator` ...).
- KB invariants: the frozen-grammar test (**B**) now enumerates **12** `AssertionType` values (adds `ordered`); the `afterAction-links-a-real-action-id` invariant (**F**) passes for the sort scenario, confirming `search-pos-sort.select.sort_dropdown` matches its materialized action id.

Deliberate ratchet update (reviewed, not a silent break)

Sprint 2D installed a guard: “no non-authentication module may author action/assertion templates until auth is 100%.” This sprint deliberately authors ONE search-category scenario, so both guards were updated with an **explicit, commented allow-list** of a single entry (`search/search-pos-sort`) rather than being weakened. Everything else the guards protect stays enforced.

Pre-existing failures NOT caused by this sprint (disclosed)

On `main` , three KB-invariant assertions already fail because the `auth-reset-*` scenarios were shipped without authored templates (auth is not actually at 100%). These are unrelated to this sprint and were red before any change here. Left as-is; flagged for a future auth-completion task.

5. Honest gap — ordering comparator is lexical for now

Ordering is now **verified**, not just described, in both views. The one remaining scoped item: the Script-Gen comparator is **lexical** (`localeCompare`), which is correct for textual dimensions like `name` . A numeric dimension (e.g. `price`) needs a numeric comparator. No such

scenario is authored yet, so this is deliberately deferred — internal engineering backlog, **not** a customer-facing placeholder or TODO, and nothing degraded ships. When a numeric ordering scenario is authored, the comparator selection keys off `orderBy` / a dimension hint; the semantic model already carries enough to make that choice.

6. Scope boundaries (as agreed)

- **Classification is out of scope.** “Sort products” still classifies under `search` (there is no `sorting` category), and the classifier is keyword-scored — the word “order” in a requirement can pull it toward `checkout`. We did **not** touch classification code this sprint.
 - **The canonical model stays neutral.** No renderer wording lives in the graph; adding a new view (BDD, Java, Python ...) never requires touching the model.
 - **PR #320 (`requirement-capability-detector`) stays parked.** This branch is based on `main`, not on that branch.
-

7. What Phase 4 becomes: authoring, not engineering

The engine work is done. Adding the next capability (Search, Filter, Upload, CRUD, Approval, Checkout, Payment ...) is now purely **authoring** a KB entry — `semantics` + `actionTemplate` + `assertionTemplate`, all **semantic**, no prose. Every renderer (manual, BDD, Playwright, Java, Python, Coverage, RTM, Executive-Summary) derives its own wording from the one source. Each new capability should be authored, reviewed on ONE scenario end-to-end (as done here), then expanded.

8. Files changed

File	Change
src/graph/scenario-graph.ts	ScenarioAction stays purely semantic (no prose); AssertionType += ordered ; ScenarioAssertion += collection / direction / orderBy ; OrderDirection type
src/engines/qa-knowledge-engine.ts	template interfaces mirror the neutral model; authored search-pos-sort (Sorting) with an ordered assertion, no prose
src/graph/scenario-graph-builder.ts	materializers thread the semantic ordering fields (no prose fields)
src/graph/canonical-manual-renderer.ts	NEW pure renderer: derives all wording from semantics, incl. the ordered sentence
src/graph/scenario-graph-adapters.ts	toTestCaseLab() renders from canonical when present; legacy fallback
src/script-gen/script-gen-engine.ts	emitGraphAssertionLines handles ordered with a real Playwright ordering check (no silent drop)
tests/unit/qa-knowledge-actions-invariant.test.ts	ratchet allow-list (reviewed)
tests/unit/qa-knowledge-assertions-invariant.test.ts	ratchet allow-list; frozen-grammar test now enumerates 12 types
scripts/canonical-render-proof.ts	NEW manual before/after + zero-regression proof harness
scripts/canonical-ordered-scriptgen-proof.ts	NEW Script-Gen ordering-emission proof harness